Your answer NOT SUBMITTED

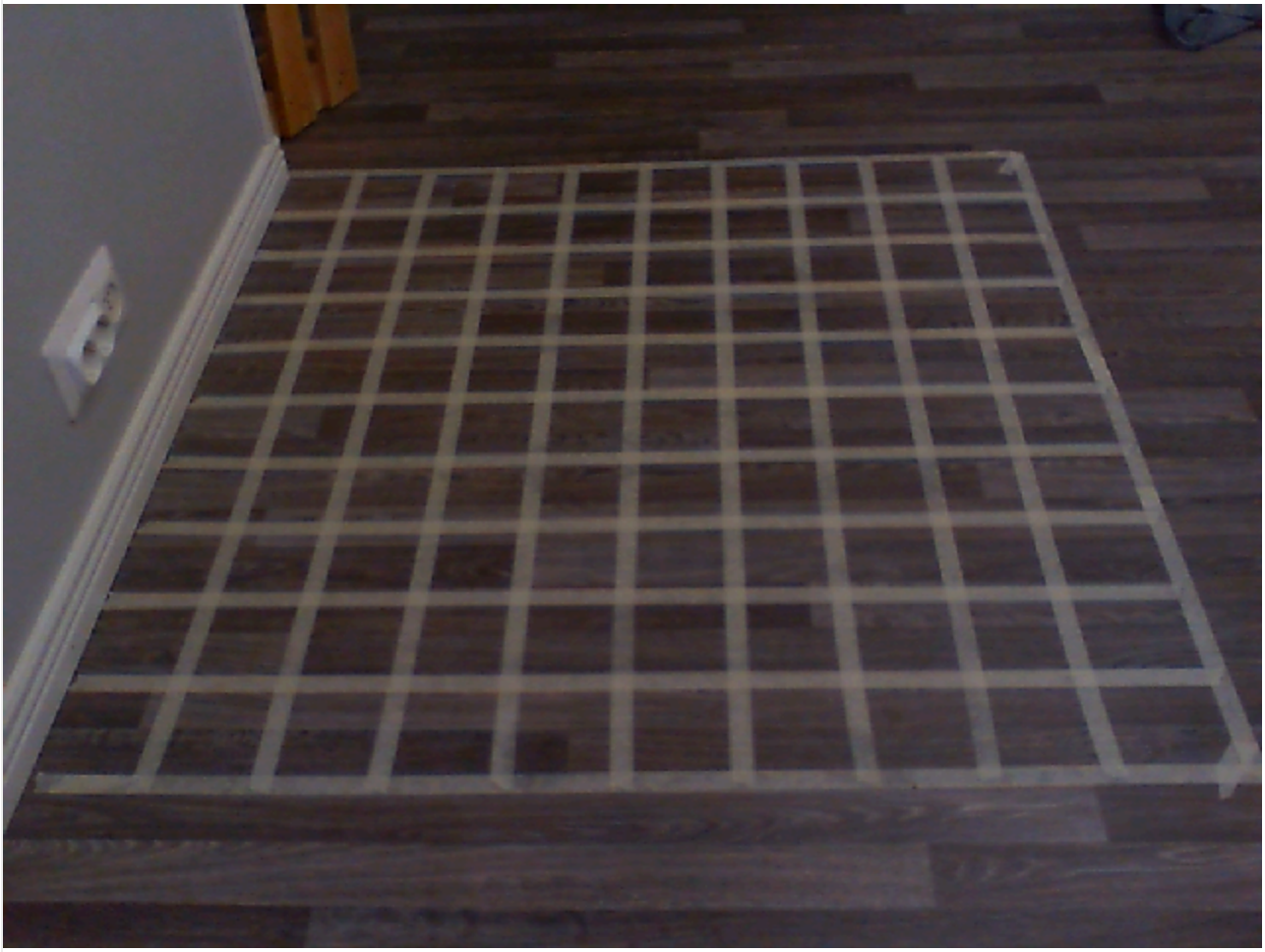
Assignment 2 file(s)

Description:

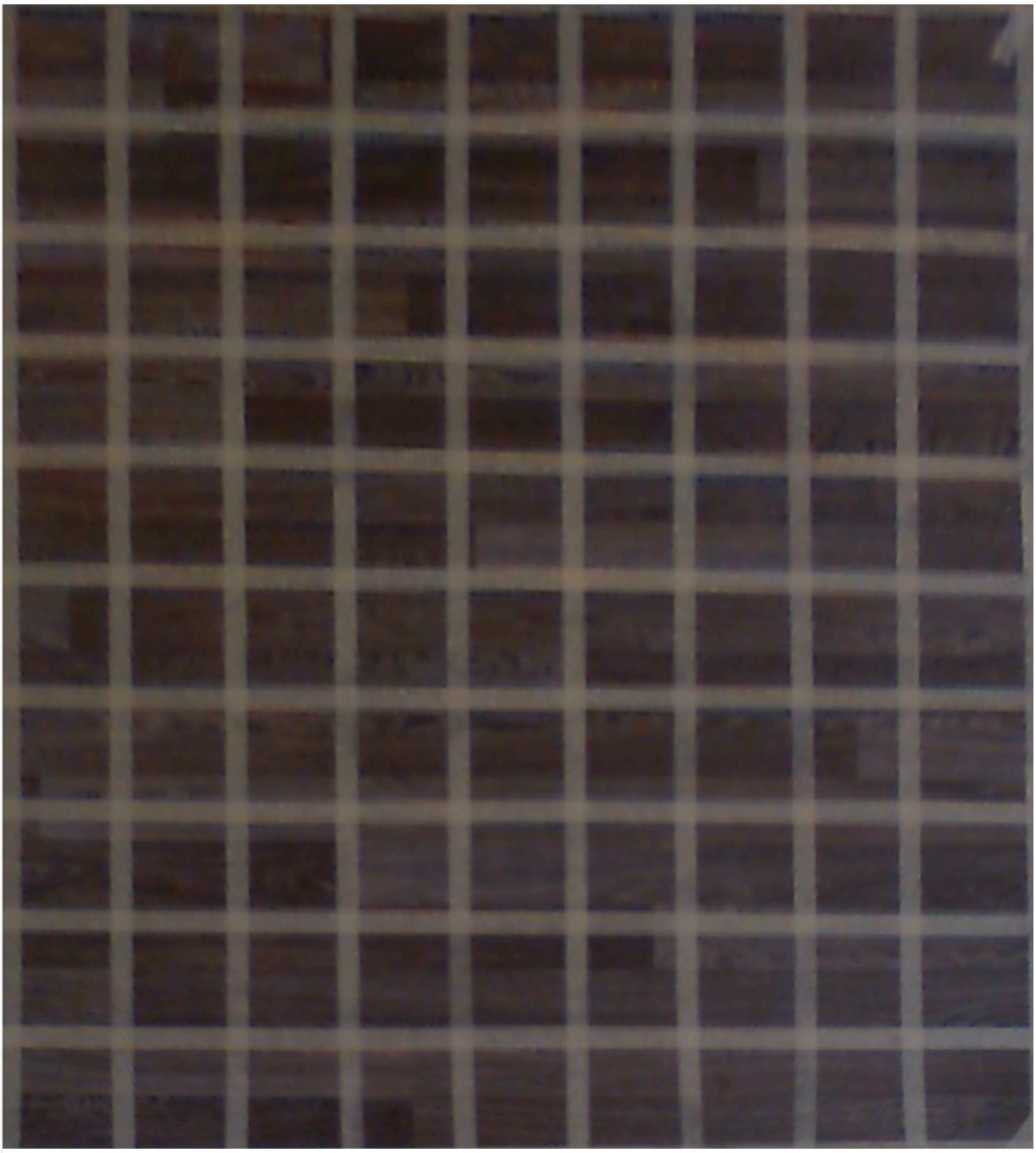
You are given a photo of a taped grid on a floor (see attached sample). Write a Python program that:

1. **Detects all grid intersection points (corners)** in the image,
2. **Finds the four outer corners** of the grid,
3. **Performs perspective rectification** (homography) so the grid appears as a top-down, rectangular view, and
4. **Rotates/aligns** the final image so that the grid's horizontal and vertical lines align with the image axes.

image1_1.png:



Perspective corrected image:



Learning Objectives

- Design an end-to-end vision pipeline: preprocessing → feature detection → geometry → transformation.
- Apply corner and line detection (Canny/Hough/Harris/GoodFeaturesToTrack) and **homography** (cv2.getPerspectiveTransform / cv2.findHomography).
- Handle noise/illumination (thresholding, morphology, normalization).
- Visualize and evaluate results.

Minimum Requirements

1. **Input**: a single grid image (image1_1 **.** `png`).

2. **Corners**: detect **all internal intersections** (coordinates in image space) and overlay them on the image.
3. **Outer corners**: identify the **four outer corners** (TL, TR, BR, BL) and visualize them.
4. **Perspective rectification**: compute the homography and output a **rectified** top-down image.
5. **Axis alignment**: ensure the rectified image is aligned so grid lines are horizontal/vertical (apply a small extra rotation if needed).

Suggested Workflow (guidance)

1. **Preprocessing**

    - Optional downscale, grayscale, smoothing (GaussianBlur).
    - Contrast improvement (CLAHE) or illumination compensation.
    - Thresholding (global/adaptive) and/or Canny edges.
    - Morphology (open/close) to suppress floor texture noise.

2. **Detect grid structure** (two alternative strategies—use one or combine):

    - **A. Lines → Intersections**

        - HoughLinesP → split lines into near-vertical and near-horizontal by angle.
        - Merge collinear fragments into consolidated lines (clustering/averaging).
        - Compute **intersections** for each vertical × horizontal pair; keep those within the image bounds.

    - **B. Corners → Row/Column Grouping**

        - Detect corners via goodFeaturesToTrack / Harris on a thresholded/edge image.
        - Cluster points into rows and columns (e.g., DBSCAN/k-means on 1D x/y coordinates) → infer the grid intersections.

3. **Find the outer four corners**

    - Take extreme intersections (min sum → TL, max sum → BR, min (x−y) → TR, max (x−y) → BL), **or**
    - Use convex hull + polygon approximation (approxPolyDP) to estimate the grid boundary.
    - Order corners as TL, TR, BR, BL.

4. **Perspective rectification**

    - Choose target size: e.g., `[W × H] = [num_cols * cell_size, num_rows * cell_size]`.
    - Compute `M = cv2.getPerspectiveTransform(src_pts, dst_pts)` and apply `cv2.warpPerspective`.

5. **Axis alignment**

- Measure residual skew of horizontal lines (e.g., HoughLines again) and rotate a few degrees using `cv2.getRotationMatrix2D` + `cv2.warpAffine`; or enforce alignment via your choice of destination points.

6. **Results**
   - Show intermediate outputs: `edges`, `lines`, `corners`, `rectified`, `rectified_rotated`.

Deliverables

1. **Short Video report**
   - Description of the processing pipeline
   - Discussion of accuracy and limitations

Grading Criteria (6 points total)

Criterion	Points
Functionality & Accuracy (4 pts): inner intersections, correct outer corners, valid rectification.	4 pts
Code Quality (2 pts): modular, readable, commented.	2 pt
Total	**6 pts**

image1_1.png ▪▪▪

pixel_values.csv ▪▪▪